

1 Objetivos

A parte prática da disciplina de Segurança e Confiabilidade pretende familiarizar os alunos com alguns dos problemas envolvidos na programação de aplicações distribuídas seguras, nomeadamente a gestão de chaves criptográficas, a geração de sínteses seguras, cifras e assinaturas digitais, e a utilização de canais seguros à base do protocolo TLS. O trabalho a desenvolver na disciplina será realizado utilizando a linguagem de programação Java e a API de segurança do Java, e é composto por duas fases.

Na primeira fase desenvolveu-se um sistema simplificado de armazenamento de ficheiros, designado por **mySharing**, onde o utilizador recorre a um servidor central para armazenar e, se desejado, partilhar os seus ficheiros. A arquitetura cliente-servidor que se pretende implementar é representada na Figura 1.

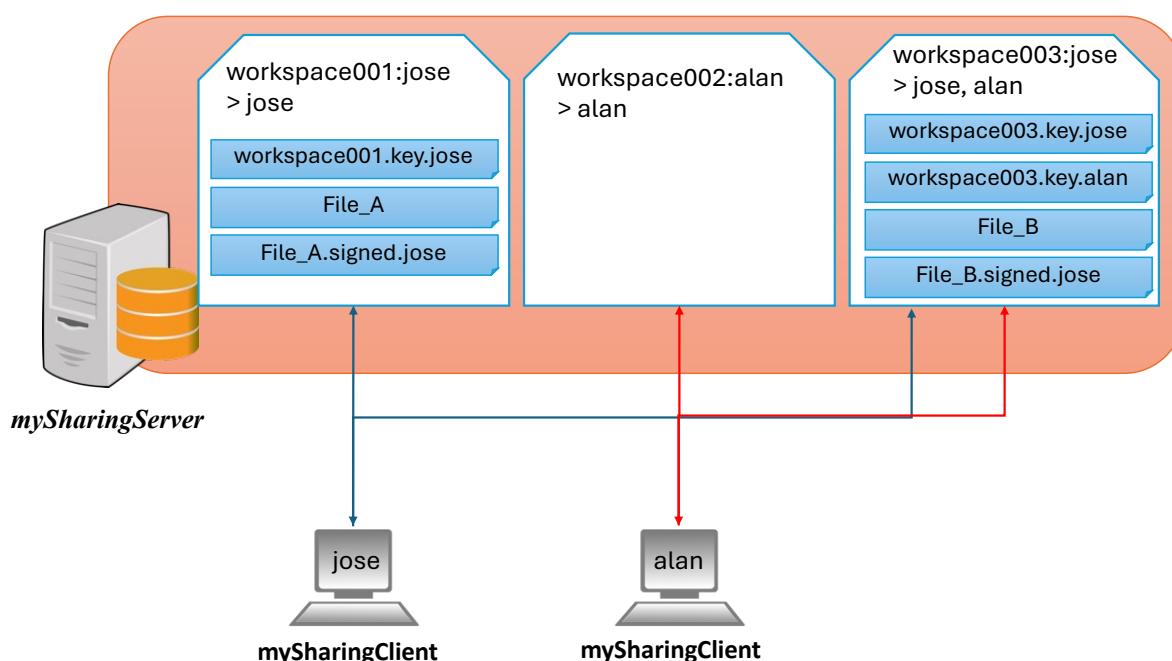


Figura 1 Diagrama da arquitetura do sistema **mySharing**, com um servidor e múltiplos clientes.

A primeira fase do projeto teve como objetivo fundamental a construção de uma aplicação distribuída, tendo sido concretizadas as funcionalidades oferecidas pelo serviço. Nesta segunda fase do projeto serão considerados requisitos de segurança, para que as interações e o sistema no seu todo sejam seguros. As várias funcionalidades definidas na primeira fase serão mantidas, embora a sua concretização tenha em alguns casos de ser adaptada para satisfazer os requisitos de segurança.

2 Modelo de sistema e definições de segurança

A arquitetura do sistema segue o que foi definido na primeira fase do projeto. O sistema deve concretizar os seguintes mecanismos de segurança:

- A. O servidor deve proteger a **confidencialidade das passwords**. Com este objetivo, as passwords devem ser armazenadas utilizando mecanismos baseados em algoritmos de sínteses e *salts*. O ficheiro dos utilizadores deve ser um **ficheiro de texto**, onde cada linha contém o *user-id* e o hash da (*password* + *salt*) e o *salt*.

Exemplo: <user-id>:<hash(password + salt)>:<salt>

jose:efnsuwiBuk+XDCu39p1D6Wn6/kUnhWQZ7/eokKUJmUQ=:xQ+KUF46offqxaQ8pIz09Q==

- B. O servidor deve verificar a **integridade dos ficheiros de utilizadores e workspaces**. Para tal, o cada ficheiro deve ser protegido com um MAC. O cálculo deste MAC utiliza uma chave simétrica calculada a partir da password do sistema *mySharing* que é pedida ao utilizador quando se **inicia a execução do servidor**.

No início da sua execução, o servidor deve usar o MAC para verificar a integridade dos ficheiros. Se o MAC estiver errado, o servidor deve imprimir um aviso e terminar imediatamente a sua execução. Se não há MAC para os ficheiros, o servidor deve imprimir um aviso e perguntar ao utilizador (administrador do servidor *mySharing*) se pretende calcular o MAC. O MAC deve **ser verificado em todos os acessos aos ficheiros de utilizadores e workspaces, e atualizados caso o(s) ficheiro(s) seja(m) alterado(s)**.

Os MACs devem ser guardados em ficheiros próprios apenas para este efeito (um para utilizadores e outro para os workspaces).

- C. Na comunicação entre o cliente e o servidor pretende-se garantir a **autenticidade do servidor** (um atacante não deve ser capaz de fingir ser o servidor e assim obter a password de um utilizador) e a **confidencialidade** da comunicação entre cliente e servidor (um atacante não deve ser capaz de escutar a comunicação). Nesta segunda fase do projeto, **todas as comunicações serão feitas através de sockets seguros** (protocolo TLS/SSL) **com autenticação unilateral**.

As chaves privadas estarão armazenadas em **keystores** (uma por cada utilizador e uma para o servidor) protegidas por passwords. Para além disso, e dado que todos os utilizadores terão necessidade de utilizar as chaves públicas uns dos outros e do servidor, os certificados de chave pública (auto-assinados) do servidor e dos clientes devem ser adicionados a uma **truststore** a ser usada por todos. Ou seja, o servidor e cada cliente terão uma cópia da **truststore** já contendo todos os referidos certificados.

- D. Os ficheiros dos utilizadores armazenados no servidor devem ser protegidos de eventuais ataques que possam ocorrer na máquina servidora, nomeadamente que tenham em vista **observar o seu conteúdo ou alterá-lo**.

A **integridade** e a **autenticidade** dos ficheiros devem ser verificáveis com assinaturas digitais. Assim, quando um utilizador envia um ficheiro para o servidor, o ficheiro deve ser previamente assinado com a chave privada do utilizador.

A **assinatura** deve ser guardada num ficheiro adicional com extensão *signed.user-id*, o qual também é enviado e armazenado no servidor. Quando é feito o download de um ou de múltiplos

ficheiros, as assinaturas também devem ser descarregadas e verificadas no cliente. O *user-id* identifica o utilizador que gerou a assinatura.

Sugere-se que, em ambiente de desenvolvimento, cada utilizador tenha uma keystore própria, designada por *keystore.user-id*, na qual são guardadas a chave privada e pública do utilizador.

A **confidencialidade** dos ficheiros é assegurada através de criptografia híbrida. Os ficheiros enviados para o servidor devem ser previamente cifrados. Note que a confidencialidade deve ser **fim-a-fim**, ou seja, apenas os utilizadores pertencentes ao(s) workspace(s) aos quais o dispositivo pertence podem ler os dados. Isto deve ser assegurado mesmo nos casos em que o workspace não seja partilhado.

Isto significa que os ficheiros partilhados poderão ser decifrados por todos os utilizadores pertencentes ao mesmo workspace. Utilizadores não pertencentes a um dado workspace não poderão aceder e/ou decifrar tais ficheiros, assim como o servidor também não poderá.

Para assegurar a confidencialidade dos ficheiros partilhados, será necessário definir uma chave para cada workspace (chave do workspace). Esta chave, criada pelo *Owner* do workspace, será partilhada apenas com os utilizadores do workspace por meio de criptografia assimétrica. Para isso, os seguintes pontos devem ser observados:

- 1) A chave do workspace (simétrica) é criada pelo Owner do workspace com base numa password e num *salt* (aleatório) – ou seja, num esquema de Password-based Encryption (PBE). Sugere-se que o Owner guarde a chave gerada num ficheiro de texto cifrado (com a própria chave pública) dentro do workspace.

O comando para criar um workspace deverá ser alterado para:

CREATE <ws> <password>

Onde **<password>** representa a password escolhida para o workspace.

- 2) Cada vez que o Owner de um workspace adiciona um novo utilizador a esse workspace, o Owner deverá criar um ficheiro de texto com a chave do workspace, cifrá-lo com a chave pública do utilizador a adicionar e enviá-lo para o respetivo workspace. O ficheiro deve ter o seguinte nome *<ws>.key.<user-id>*, em que *<ws>* é o nome do workspace e *<user-id>* é o identificador do utilizador.

Ou seja, o método que implementa a função de adição de utilizador a um workspace deverá ser alterado para contemplar a criação de um ficheiro de texto com a chave do workspace, cifrá-lo com a chave pública do utilizador a adicionar e enviá-lo para o respetivo workspace.

- 3) Sempre que um utilizador pretender fazer um upload de ficheiros para um workspace, o servidor deverá verificar se o utilizador tem permissão de acesso. Em caso afirmativo, o servidor enviará a chave do workspace respetivo (cifrada com a chave pública do utilizador). Ao receber a chave cifrada do workspace, o utilizador terá de decifrá-la usando a sua chave privada. Com base na chave do workspace, o utilizador deve cifrar todos os ficheiros previamente e de seguida enviá-los para o workspace.
- 4) Sempre que um utilizador pretender fazer um download, o servidor deverá verificar se o utilizador tem permissão de acesso. Em caso afirmativo, o servidor enviará os ficheiros solicitados juntamente com a chave do workspace respetiva (cifrada com a chave pública do utilizador). Ao receber a chave cifrada do workspace, o utilizador terá de decifrá-la usando a sua chave privada, e, em seguida, o utilizador decifrará os ficheiros usando a chave do

workspace.

NOTA: Toda criptografia assimétrica no projeto deve usar RSA com chaves de 2048 bits. A criptografia simétrica deve ser efetuada com AES e chaves de 128 bits. Para as sínteses, deve ser usado um algoritmo seguro.

3 Entrega

3.1 Trabalho

Dia **29 de abril**, até às 23:59 horas. O código desta segunda (e última) fase do trabalho deve ser entregue de acordo com as seguintes regras:

- A segunda fase do projeto deve ser realizada mantendo os grupos da primeira fase.
- Para entregar o trabalho, é necessário criar um **ficheiro zip** com o nome **SegC-grupoXX-proj1-fase2.zip**, onde **XX** é o número do grupo, contendo:
 - ✓ o código do trabalho;
 - ✓ conjunto de chaves, *keystores*, certificados (ficheiros de formato **cert**) e *truststore*;
 - ✓ os ficheiros jar (cliente e servidor) para execução do projeto;
 - ✓ um readme (txt) indicando **como compilar** e **como executar** o projeto, indicando também as limitações do trabalho.
- O ficheiro zip é submetido através da atividade disponibilizada para esse efeito na página da disciplina no Moodle. Apenas um dos elementos do grupo deve submeter. Se existirem várias submissões do mesmo grupo, será considerada a mais recente.
- Não serão aceites trabalhos enviados por email. Se não se verificar algum destes requisitos o trabalho é considerado não entregue.

3.2 Autoavaliação de contribuições

Dia **2 de maio**, até às 23:59 horas. Cada aluno preenche no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo na 2ª fase do projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário sofrem uma penalização na nota de 10%.
- Caso existam assimetrias entre as contribuições de cada elemento do grupo ou entre as autoavaliações de cada elemento do grupo, estas serão analisadas durante a discussão do trabalho e poderão levar à atribuição de notas individuais diferentes para cada elemento do grupo.